

Homeaway From Home!

Object-Oriented Programming

1st Project, version 1.2 – 2025-04-09

Added text is presented **in red**

Changed command messages are printed in **green**

Important remarks

Deadline until 23h55 (Lisbon time) of May 2nd, 2025.

Team this project is to be MADE BY GROUPS OF 2 STUDENTS.

Deliverables: Submission and acceptance of the source code to **Mooshak** (score > 0). See the course website for further details on how to submit projects to Mooshak. Partial checkpoints submissions will be expected and will add to the weekly assignments component (see information on Moodle, shortly).

Recommendations: We value the documentation of your source code, as well as the usage of the best programming style possible and, of course, the correct functioning of the project. Please carefully comment both interfaces and classes. The documentation in classes can, of course, refer to the one in interfaces, where appropriate. Please comment on the methods explaining what they mean and defining preconditions for their usage. Students may and should discuss any doubts with the teaching team, and with other students, but may not share their code with other colleagues. This project is to be conducted with full respect for the Code of Ethics of NOVA University, Lisbon, available on the course website.

Checkpoints: To encourage students to start early, there will be two intermediate checkpoints in this project. The first one, due on April 14 until 17h00 (Lisbon time), is to check that your command interpreter is working correctly. The second one, due on April 21 until 17h00, is to check that the program correctly creates all kinds of students and services. Both checkpoints will be treated as regular mini-assignments, similar to those mini-assignments that students have been doing in the labs. The sets of tests in these two checkpoints are sublists of the overall tests set. See the list of tests in Moodle. Both team members should submit these checkpoints.

1 Development of the application *HomeAway From Home*

1.1 Problem description

The goal of this project is to develop an application that supports international students in a Campus oriented town by providing information about services that are useful to them. These services are location-based (which means they may be referred to as locations) and may be of various kinds, which will be defined by the particular application. We will provide you with examples for Lisbon's Metropolitan area, which includes Almada, Monte da Caparica and Costa

de Caparica, where several universities and Higher Education Schools exist, but the application should be useful for any such a town.

For the sake of this assignment, the type of services will be *Eating* (such as low budget restaurants or canteens) and *Lodging* (rooms which can be let by students and in student residences) as well as *Leisure* (Entertainment/Cultural) because student life is not just about saving money. Be aware that the system should not be limited to these types and should be prepared for services' extensions.

The application should be geographically-oriented, which means that the town is defined by a bounding box (rectangle) and should only accept new services included in that bounding box.

Each service will always include a name and a point location, which is composed of latitude and longitude (for simplicity, in our system, latitude and longitude are integer (long) numbers that represent the real decimal values multiplied by 1 000 000 000). Each type of service will include additional information. An *Eating* service will include the cost of the daily student menu. The *Lodging* type of service will include the weekly cost of the room, for students. The *Leisure* type will include the ticket price and the percentage of student discount. **Be aware that the system's input and output does only handle integer values (although you may want to use float values for internal computations).**

The system should also enable diverse types of students. For simplicity, we will consider *Bookish* students (mainly concerned with studying and visiting/attending *Leisure* sites and/or events), *Outgoing* students (who are mainly concerned with eating out and visiting the town) and *Thrifty* students (whose main concern is to save money and live on a budget). At a particular moment, the location of a student may be their current home (a lodging service) or any services known by the system. A bookish student collects information about the leisure places that they have visited in the town. Outgoing students collect information about all the locations/services they have visited, with no restrictions. Thrifty students are always trying to find the most inexpensive place to eat and sleep. They always seek for the most inexpensive canteen/restaurant and residence/hostel in town and will only move from their current home if they find a cheaper service than the one they currently use.

Students may evaluate services that they have visited, stayed in or ate at. Evaluations are submitted as stars and each service can provide its updated evaluation, which is the integer rounded result of the average of all the evaluations received. A freshly created service is given 4 stars. Listings of locations ordered by star evaluation may be requested by the system. A particular student may also require to know the most relevant service (for them) of particular type.

The system will know all the services inside the defined bounding box and all registered students. Students should only go or move to locations inside the bounding box.

2 Commands

In this section we present all the commands that the system must be able to interpret and execute. In the following examples, we differentiate *text written by the user* from the feedback written by the program in the console. You may assume that the user will make no mistakes when using the program other than those described in this document. In other words, you only need to take care of the error situations described here, in the exact same order as they are described.

Commands are case insensitive. For example, the *exit* command may be written using any combination of upper and lowercase characters, such as *EXIT*, *exit*, *Exit*, *exIT*, and so on. In the examples provided in this document, the symbol ↵ denotes a change of line.

If the user introduces an unknown command, the program must write in the console the message `Unknown command. Type help to see available commands.` For example, the non existing command `someRandomCommand` would have the following effect:

```
someRandomCommand↵
Unknown command. Type help to see available commands.↵
```

If there are additional tokens in the line (e.g. parameter for the command you were trying to write, the program will try to consume them as commands, as well. So, in the following example, `someRandom Command` would be interpreted as two unknown commands: `someRandom` and `Command`, leading to two error messages.

```
someRandom Command↵
Unknown command. Type help to see available commands.↵
Unknown command. Type help to see available commands.↵
```

Several commands have arguments. Unless explicitly stated in this document, you may assume that the user will only write arguments of the correct type. However, some of those arguments may have an incorrect value. For that reason, we need to test each argument exactly by the order specified in this document. Arguments will be denoted **with this style**, in their description, for easier identification. Also, for any String arguments in the commands, assume they are case-sensitive. So, “Coffee Shop” and “coffee shop” would be two different services, for instance.

2.1 `exit` command

Terminates the execution of the program. This command does not require any arguments. The following scenario illustrates its usage.

```
exit↵
Bye!↵
```

This command always succeeds.

2.2 `help` command

Shows the available commands. This command does not require any arguments. The following scenario illustrates its usage.

help↵

bounds - Defines the geographic bounding rectangle of the system↵

eating - Adds a new service of type eating to the system↵

lodging - Adds a new service of type lodging to the system↵

leisure - Adds a new service of type leisure to the system↵

services - Displays the list of services in the system↵

student - Adds a student to the system↵

students - Lists all the students in the community↵

leave - Removes a student from the system↵

go - Changes the location of a student to a service, or home↵

move - changes the home of a student↵

star - evaluates a service↵

where - locates a student↵

visited - lists locations visited by one student↵

ranking - lists services ordered by star↵

ranked - lists services of a certain type, with a specific star evaluation↵

find - Finds the most relevant service of a certain type, for a specific student↵

help - shows the available commands↵

exit - terminates the execution of the program↵

This command always succeeds. When executed, it shows the available commands.

2.3 bounds command

Defines the geographic bounding rectangle of the system. The command receives, as arguments, the **name** of the area to be considered in the system and its bounding rectangle, composed of the **latitude** and the **longitude** of the top left and bottom right corners of this bounding rectangle (in this order). The values will be integers. This command always succeeds but it removes any data that the system currently stores. The command returns the feedback message (<bounds name> created.).

In the example, we define the bounding rectangle for the called **Lisbon Area**.

```
bounds 38848911991 -9497899738 38461247511 -8885274394 Lisbon Area↵  
Lisbon Area created.↵
```

In the context of this command, two situations of exception may occur:

1. The bounds command has not yet been submitted. In this case, the system cannot really do anything so, the execution of any command will generate the message (System bounds not defined.).
2. If the bounds rectangle defined is not valid - if the top latitude is lower **or equal to** the bottom latitude and the right longitude is lower **or equal to** the left longitude - (it is not possible to include any services in the defined rectangle), it will not be possible to add any information to the system. In this case, the system will return the message (Invalid bounds.) and the bounds value will have to be re-submitted.

2.4 eating command

Adds a new service of type eating to the system. The command receives, as arguments, the **latitude** and the **longitude** of the service, the **price** of the student menu and the **name** of the service. When successful, the program will output the feedback message (<service name> added.). In the example, we create a new eating service called **Akatsuki Caparica** at (38.639378483, -9.234400975) with the price of the student menu at **8** euros. There is no upper limit to the number of services (of any type) one can create in the system.

```
eating 38639378483 -9234400975 8 Akatsuki Caparica↵  
Akatsuki Caparica added.↵
```

The following errors may occur:

1. If the **studentMenu** price is less or equal to **0**, the error message is (Invalid menu price!).
2. If the service location is outside the bounding rectangle defined for the system, the error message is (<name> location invalid!).
3. If the service **name** already exists in the system, the error message is (<name> already exists!).

The following example illustrates these error messages.

```
eating 38639378483 -9234400975 -200 restaurant new↵  
Invalid menu price!↵  
eating 40848911991 -6497899738 12 restaurant new↵  
restaurant new location invalid!  
eating 38639378483 -9234400975 8 Akatsuki Caparica↵  
Akatsuki Caparica already exists!↵
```

2.5 lodging command

Adds a new service of type lodging to the system. The command receives, as arguments, the **latitude** and the **longitude** of the service, the price of the **studentRoom** and the **name** of the service. When successful, the program will output the feedback message (<service name> added.). In the example, we create a new lodging service called **Frausto da Silva Residency** at (38.665184410, -9.208892510) with the monthly price of the student room at **251** euros. There is no upper limit to the number of services (of any type) one can create in the system.

```
lodging 38639378483 -9234400975 251 Frausto da Silva Residency↵  
Frausto da Silva Residency added.↵
```

The following errors may occur:

1. If the **studentRoom** price is less or equal to **0**, the error message is (Invalid room price!).
2. If the service location is outside the bounding rectangle defined for the system, the error message is (<name> location invalid!).

3. If the service **name** already exists in the system, the error message is (<name> already exists!).

The following example illustrates these error messages.

```
lodging 38639378483 -9234400975 -200 New Residency↵
Invalid room price!↵
lodging 40848911991 -6497899738 300 New Residency↵
New Residency location invalid!
lodging 38639378483 -9234400975 251 Frausto da Silva Residency↵
Frausto da Silva Residency already exists!↵
```

2.6 leisure command

Adds a new service of type leisure to the system. The command receives, as arguments, the **latitude** and the **longitude** of the service, the **ticket** price, the percentage of the student **discount** and the **name** of the service. When successful, the program will output the feedback message (<service name> added.). In the example, we create a new leisure service called **Joaquim Benite Theater** at (38.676672199, -9.159911717) with the ticket price of **12** euros and the student discount of **20**%. There is no upper limit to the number of services (of any type) one can create in the system.

```
leisure 38676672199 -9159911717 12 20 Joaquim Benite Theater↵
Joaquim Benite Theater added.↵
```

The following errors may occur:

1. If the **ticket** price is less or equal to **0**, the error message is (Invalid ticket price!).
2. If the student **discount** is less than **0** or larger than **100**, the error message is (Invalid discount!).
3. If the service location is outside the bounding rectangle defined for the system, the error message is (<name> location invalid!).
4. If the service **name** already exists in the system, the error message is (<name> already exists!).

The following example illustrates these error messages.

```
leisure 38676672199 -9159911717 -15 20 New Theater↵
Invalid ticket price!↵
leisure 38676672199 -9159911717 15 150 New Theater↵
Invalid discount!
leisure 40848911991 -6497899738 15 10 New Theater↵
New Theater location invalid!
leisure 38676672199 -9159911717 12 20 Joaquim Benite Theater↵
Joaquim Benite Theater already exists!↵
```

2.7 services command

Displays the list of services in the system. The command does not receive any arguments and always succeeds. If there are no services to list, the program will output the feedback message (No services yet!). Otherwise, it lists all services, in order of insertion, each line presenting the service name, its type, latitude and longitude (<service name>: <type> <latitude> <longitude>.). In the example, there are three services available in the system.

```
services↵
Akatsuki Caparica: eating 38639378483 -9234400975.↵
Frausto da Silva Residency: lodging 38639378483 -9234400975.↵
Joaquim Benite Theater: leisure 38676672199 -9159911717.↵
```

2.8 student command

Adds a student to the system. The command receives as arguments the **type** of student (bookish, outgoing or thrifty), their **name**, and the name of the **lodging** where they are living currently. When successful, the program will output the feedback message (<name> added.). We assume that the student's location, at creation, is their lodging service. In the example, we create a new bookish student called **Neil Perry**.

```
student bookish↵
Neil Perry↵
Frausto da Silva Residency↵
Neil Perry added.↵
```

The following errors may occur:

1. If the student's **name** already exists in the system, the error message is (<name> already exists!).
2. If the **lodging** does not exist in the system, the error message is (Lodging <lodging> does not exist!).

The following example illustrates these error messages.

```
student bookish↵
Neil Perry↵
Frausto da Silva Residency↵
Neil Perry already exists!↵
student bookish↵
Todd Anderson↵
Welton Academy Residency↵
Lodging Welton Academy Residency does not exist!↵
```

2.9 leave command

Removes a student from the system. The command receives as argument the student's **name**. When successful, the program will output the feedback message (<name> has left.). In the example, we remove student **Neil Perry**.

```
leave Neil Perry↵
Neil Perry has left.↵
```

The following errors may occur:

1. If the student's **name** does not exist in the system, the error message is (<name> does not exist!).

The following example illustrates this error message, the attempt to remove a non-existent student from the system.

```
leave Jack Keating↵
Jack Keating does not exist!↵
```

2.10 students command

Lists all the students in the community. The command does not receive any arguments and always succeeds. If there are no students to list, the program will output the feedback message (No students yet!). Otherwise, it lists all the students, in order of insertion, each line presenting the students's name, their type and their current location's name. In the example, there are three students in the system.

```
students↵
Neil Perry: bookish at Frausto da Silva Residency.↵
Knox Overstreet: outgoing at Joaquim Benite Theater.↵
Todd Anderson: thrifty at NOVA FCT Canteen.↵
```

2.11 go command

Changes the location of a student to a service, or home. The command receives as arguments the student's **name** and the name of the **location** where the student is going. This location can either be a service in the system or the student's home. Home is also, ultimately, a lodging service in the system. Bookish students store the leisure services they have visited and outgoing students store every service they have visited. Thrifty students do not store visited locations. When successful, the program will output the feedback message (<student name> is now at <location name>.). If the student is thrifty, and the command is moving their location to a more expensive service than the one they use, the command is accepted but the student is warned of their mishap (<student name> is distracted!).

We provide two examples. In the first example, **Neil Perry** goes to his home. In the second example, **Todd Anderson** goes to eat at **Akatsuki Caparica**, which is not their less expensive known location.

```
go Neil Perry↵
home↵
Neil Perry is now at home.↵
go Todd Anderson↵
Akatsuki Caparica↵
Todd Anderson is now at Akatsuki Caparica. Todd Anderson is distracted!↵
```


The following errors may occur:

1. If the student's **name** does not exist in the system, the error message is (<student name> does not exist!).
2. If the name of the **location** where the student is going is not known to the system, the error message is (Unknown <location name>!). Of course, this will not happen with the word home, as all students must have a home.
3. If the student decides to go to the location where they already are, the command has no effect but returns the error message (Already there!).

The following example illustrates these error messages.

```
go Charlie Dalton↵
Akatsuki Caparica↵
Charlie Dalton does not exist!↵
go Neil Perry↵
The Cave↵
Unknown The Cave!↵
go Neil Perry↵
home↵
Already there!↵
```

2.12 move command

Changes the home of a student if that is acceptable. The command receives as arguments the student's **name** and the name of the **lodging** service to become the student's new home. When successful, the student's home becomes the lodging service submitted and the student's current location also becomes the location of this service. The program will output the feedback message(**Lodging** <lodging name> is now <student name>'s home. <student name> is home.). In the example, **Neil Perry** moves to **Egas Moniz University Residence**.

```
move Neil Perry↵
Egas Moniz University Residence↵
Egas Moniz University Residence is now Neil Perry's home. Neil Perry is home.↵
```

The following errors may occur:

1. If the student's **name** does not exist in the system, the error message is (<name> does not exist!).
2. If the **lodging** service name does not exist in the system, the error message is (**Lodging** <lodging name> does not exist!).
3. If the student's home is already the one provided in the command, the system should do nothing and only provide the message (That is <name>'s home!).
4. If a thrifty student tries to move to a same price or more expensive lodging, the error message is (Move is not acceptable for <name>!).

The following example illustrates these error messages.

```
move John Keating↵
Egas Moniz University Residence↵
John Keating does not exist!↵
move Neil Perry↵
The Cave↵
Lodging The Cave does not exist!↵
move Neil Perry↵
Egas Moniz University Residence↵
That is Neil Perry's home!↵
move Todd Anderson↵
Egas Moniz University Residence↵
Move is not acceptable for Todd Anderson!↵
```

2.13 star command

Evaluates a service. The command receives as arguments an integer **n** between 1 and 5, representing the number of stars attributed to the service, followed by the service's **name**. Evaluations are anonymous. The system should maintain the average of the evaluation provided, including the initial one, attributed at the time of creation. When successful, the program will output the feedback message (Your evaluation has been registered!). In the example, **Egas Moniz University Residence** received a vote of **4**.

```
star 4 Egas Moniz University Residence↵
Your evaluation has been registered!↵
```

The following errors may occur:

1. If the service's **name** does not exist in the system, the error message is (<name> does not exist!).
2. If the number **n** is not between 1-5, the error message is (Invalid evaluation!).

The following example illustrates these error messages.

```
star 2 Welton Academy Residency↵
Welton Academy Residency does not exist!
star 0 Rudolph the Reindeer↵
Invalid evaluation!↵
```

2.14 where command

Locates a student. The command receives as argument the student's **name**. When successful, the program will output the feedback message (<student name> is at <location name> (<location latitude>, <location longitude>)). In the example, student **Todd Anderson** is at **Cinemas NOS Almada Forum (38661501872, -9174040540)**.

```
where Todd Anderson↵
Todd Anderson is at Cinemas NOS Almada Forum (38661501872, -9174040540).↵
```

The following error may occur:

1. If the student's **name** does not exist in the system, the error message is (<name> does not exist!).

The following example illustrates this error message, the attempt to locate a non-existent student.

```
where John Keating↵
John Keating does not exist!↵
```

2.15 visited command

Lists the locations visited and stored by one student. The command receives as argument the student's **name** and, if it is the name of a bookish or outgoing student, it lists the locations stored by them. Thrifty students do not store the locations they visited. When successful, the program will list the names of the locations visited by the student, one in each line, sorted by visiting order. In the example, leisure places visited by bookish student **Neil Perry** are listed.

```
visited Neil Perry↵
Cinemas NOS Almada Forum↵
Joaquim Benite Theater↵
```

The following errors may occur:

1. If the student's **name** does not exist in the system, the error message is (<name> does not exist!).
2. If the student has not visited any locations, the message provided will be (<name> has not visited any locations!).
3. If the student is thrifty, the message provided will always be (<name> is thrifty!).

The following example illustrates these error messages.

```
visited Jack Keating↵
Jack Keating does not exist!↵
visited Neil Perry ↵
Neil Perry has not visited any locations!↵
visited Todd Anderson↵
Todd Anderson is thrifty!↵
```

2.16 ranking command

Lists all services ordered by star. The command receives no arguments. It prints a header (Services sorted in descending order) and then lists all services in the system, sorted in descending

order, by evaluation average. The evaluation average value is an integer resulting from rounding using `Math.round(float value)`. Each printed line should include the message (`<service name>: <integer rounded evaluation average>`). For services with the same average, the order of printing should be the order by which the average was last updated by the service.

```
ranking↵
Services sorted in descending order↵
Cinemas NOS Almada Forum: 4↵
Joaquim Benite Theater: 4↵
Akatsuki Caparica: 3↵
NOVA FCT Canteen: 2↵
```

The following errors may occur:

1. If there are no services in the system, the header should be substituted by the message (No services in the system.).

The following example illustrates the error message.

```
ranking↵
No services in the system.↵
```

2.17 ranked command

List services of a certain type, with a specific star evaluation. The command receives as arguments a `n` star average and the `type` of service to be printed. When successful, the program will print a header line (`<type> services with <n> average`) followed by the list of the names of the relevant services, one per line, by order of the time the current average was obtained by the service. Each line in the output consists simply of the name of services. In the example, `leisure` services with a `4` average are printed.

```
ranked leisure 4↵
Leisure services with 4 average↵
Cinemas NOS Almada Forum↵
Joaquim Benite Theater↵
```

The following errors may occur:

1. If `n` is larger than `5` or lower than `1`, the error message is (Invalid stars!).
2. If no services of the `type` exist with `n` star average, the error message is (No `<type>` services with average!).
3. If no services of the `type` exist, the error message is (No `<type>` services!).

The following examples illustrate these error messages.

```

ranked lodging 8↵
Invalid stars!↵
ranked lodging 1↵
No lodging services with average!↵
ranked eating 1↵
No eating services!↵

```

2.18 find command

Finds the most relevant service of a certain type, for a specific student. The command receives as arguments the student's **name** and the **type** of service required. Depending on the type of the student, the system may provide the nearest service of the type (for bookish and outgoing students), or the less expensive one (for thrifty students). The system uses the Manhattan Distance for computing the distance between two locations. Namely, given two locations l^1 and l^2 , where (l_{lat}^1, l_{long}^1) and (l_{lat}^2, l_{long}^2) denote their corresponding latitude l_{lat}^* and longitude l_{long}^* coordinates, the distance should be:

$$d(l^1, l^2) = |l_{lat}^1 - l_{lat}^2| + |l_{long}^1 - l_{long}^2| \quad (1)$$

where $|\cdot|$ denotes the absolute value. In the case of ties (several services with the same distance or price), the system should provide the first service found (use order of insertion in the system). If a service, of type lodging or eating, returned by the system to a thrifty student, is less expensive than the one that they have stored, then the student must remember this information and the system should inform of that.

When the find command is successful, the program displays a message with the (**<name>**) of the service provided by the system. In the case of a thrifty student, another message may be provided, indicating the update of the stored information (**<student name> updated.**), when the returned service is less expensive than the one stored. This will always be the case, if the thrifty student is new and does not yet know (has not stored) any eating service.

We provide two successful examples. In the first example, **Neil Perry**, a bookish student, searches for the nearest **eating** service, and receives NOVA FCT Canteen.. In the second example, **Charlie Dalton**, a thrifty student, searches for the least expensive **lodging** service, and receives Frausto da Silva Residency.. The student's lodging has been updated because his previous lodging was more expensive.

```

find Neil Perry↵
eating↵
NOVA FCT Canteen.↵
find Charlie Dalton↵
lodging↵
Frausto da Silva Residency.↵
Charlie Dalton updated.

```

The following errors may occur:

1. If the student's **name** does not exist in the system, the error message is (**<name> does not exist!**).
2. If no services of the specified **type** exist in the system, the error message is (**No <type> services!**).

The following example illustrates these error messages.

```
find John Keating↵
eating↵
John Keating does not exist!↵
find Knox Overstreet↵
leisure↵
No leisure services!↵
```

3 Developing this project

Your program should take the best advantage of the elements taught in the Object-Oriented Programming course. You should make this application as **extensible as possible** to make it easier to add, for instance, new kinds of students or services.

You can start by developing the main user interface of your program, clearly identifying which commands your application should support, their inputs and outputs, and error conditions. Then, you need to identify the entities required for implementing this system. Carefully specify the **interfaces** and **classes** that you will need. You should document their conception and development using a class diagram, as well as document your code adequately, with Javadoc. You can and should reuse the package on generic arrays presented in the lectures and labs (you can find a link to source code on Moodle). You cannot change these classes and interfaces. If you need some slightly different functionality, you can either create classes that use those Arrays and iterators as instance variables and add, or constrain, the functionalities you need for your collections on those so-called wrapper classes. You can also consider extending those classes and interfaces. But do not change them. It is, in general, not a good idea to change library classes and interfaces. Also, please note that you cannot use the standard Java Collection classes in this project. Although they will be introduced in classes while you are still working on this project, we have designed this challenge for you to solve it without those classes. We will have ample opportunity of working with the standard Java Collection classes on the second project. The project should be developed using OOP content known to you on the date of publication of this document.

It is a good idea to build a skeleton of your Main class, to handle data input and output, supporting the interaction with your program. In an early stage, your program will not really do much. Remember the **stable version rule**: do not try to do everything at the same time. Build your program incrementally, and test the small increments as you build the new functionalities in your new system. If necessary, create small testing programs to test your classes and interfaces.

Have a careful look at the test files, when they become available. You should start with a really bare-bones system with the `help` and `exit` commands, which are good enough for checking whether your commands interpreter is working well, to begin with. Then, add locations/services and test their creation for different types. Test printing the service. Then, move on to adding the different kinds of students, one by one, as well as list them. Check that they are ok. Test the `leave` command too. Then move on to making students `go` places and `move` homes. You can also implement the command `where` and `visited` at this time. Once this is working you can implement the evaluation part of the application with the `star` command and the listings of evaluations (commands `ranking` and `ranked`). The `find` command is the most challenging one so we suggest you leave it for last. Step by step, you will incrementally add functionalities to your program and test them. **Do not try to develop all functionalities**

simultaneously. It is a really bad idea.

Last, but not least, **do not underestimate the effort for this project. START EARLY.**

4 Submission to Mooshak

To submit your project to Mooshak, please register your group in the Mooshak contest POO2025-TP1. The mooshak login for the group must be the concatenation of the numbers of the students that make up the group, separated by _ (the first number must be the smallest). For example, students #5678 and #5677 should have their Mooshak username **5677_5678**. **Only the projects submitted, for evaluation, through logins following the above rule will be accepted.** The name and number of students that make up the group must be inserted in the header of all submitted files, as follows:

```
/**
 * @author STUDENT1NAME (STUDENT1NUMBER) STUDENT1temail
 * @author STUDENT2NAME (STUDENT2NUMBER) STUDENT2temail
 */
```

4.1 Command syntax

For each command, the program will only produce one output. The error conditions of each command have to be checked in the exact same order as described in this document. If one of those conditions occurs, you do not need to check for the other ones, as you only present the feedback message corresponding to the first failing condition. However, the program does need to consume all the remaining input parameters, even if they are to be discarded.

4.2 Tests

We expect you to create your own tests, based on this specification. Create tests for the “happy day scenario”, and also for error situations, so that whenever you add support for a new command, you also create suitable tests for it. So, build your tests as you build your project, incrementally. This mimics what happens with the Mooshak tests. The Mooshak tests verify incrementally the implementation of the commands. They will be made publicly available on April 14th, 2025. When the sample test files become available, use them to test what you already have implemented, fix it if necessary, and start submitting your partial project to Mooshak. Do it from the start, even if you just implemented the exit and help commands. By then you will probably have more than those to test, anyway. Good luck!